



Surplus Signal Protocol

相互プロビジョニングのための最小限の連邦型調整文法

SSP v0.2仕様とは、コミュニティが資金、中央データベース、または身元監視なしに、余剰物資と必要物資を知らせることができる、ある程度判読可能なプロトコルである。

著 Björn Kenneth Holmström

with Claude, ChatGPT, DeepSeek, Mistral, Gemini and Grok

Global Governance Frameworks

余剰シグナルプロトコル v0.2

相互供給のための最小限で連合的、半可読的な調整文法

前文 – 憲法的原則

これらは法律ではない。プロトコルの精神である。違反する実装も機能しうるが、SSP が意図するものから逸脱する。

1. **局所的主権が第一**。いかなるシグナルも行動を強制しない。ノードは正当化なくいかなるシグナルも無視・拒否できる。
2. **強制的なグローバル ID なし**。識別子はデフォルトで仮名である。普遍的な本人確認を要求してはならない。
3. **中央台帳なし**。単一のグローバルなシグナルデータベースは存在しない。連合はオプトインであり、信頼ゾーンに限定される。
4. **減衰がデフォルト**。評判、可視性、感謝の指標はすべて時間とともに減衰しなければならない（デフォルト ≤ 30 日）。
5. **不透明性は設計上保存される**。正確な位置情報と検証済み ID は、明示的かつ期間限定の同意がある場合のみ開示される。
6. **フォーク権は権利**。プロトコル、実装、そしてこの前文さえも寛容な条件（CC0）でフォークできる。
7. **ハイブリッド中立性**。プロトコルはいかなる `transaction.type` も優遇・冷遇しない。貨幣と非貨幣フローは階層なく共存する。
8. **AI は助言のみ**。自動マッチング、予測、提案はすべて人間によって上書き可能でなければならない。完全自動割り当ては許可されない。
9. **強制的なグローバル指標なし**。成功は局所的に測る。強制されるグローバル KPI はない。
10. **退出は現実的でなければならない**。ノードは離脱し、過去のシグナルを削除し、自身のデータを持ち去れる（§8 参照）。

1. 導入

SSP は経済ではない。それは**主権アーキテクチャ**であり、自律的なシステム（バイオリージョン、協同組合、近隣コミュニティ、相互扶助グループ）が、支配や断片化、収奪に陥ることなく、余剰と必要の流れを調整するための方法である。

これは**プロトコルであり、プラットフォームではない**。中央データベースも、普遍的 ID も、恒久的評判も規定しない。**半可読的**に設計されており、リソースをルーティングするのに十分な可視性と、人間関係を守るのに十分な不透明性を備える。

SSP は一つの観察から出発する。現代経済が大量の無駄を生むのは、資源がないからではなく、必要と能力がお互いを見えないからである。このプロトコルは、それらを可視化するための薄い文法である——ローカルの自律性を尊重する条件のもとで。

2. 一般シグナルエンベロープ

すべての SSP シグナルは共通の外枠をまととう。

フィールド	型	必須	説明
schema_version	string	はい	"0.2"
signal_type	string	はい	登録されたシグナルタイプの一つ
id	UUID	はい	発信ノードが生成する一意識別子
node_id	string	はい	ノードごとに安定した仮名；公開鍵から導出（例：Base58 エンコードハッシュ）
timestamp	ISO 8601	はい	UTC での作成時刻
expires	ISO 8601	いいえ	シグナルが無効になる日時；デフォルト 7 日
location	object	はい*	（ offer/request の場合 ） §2.1 参照
resource	object	はい*	（ offer/request の場合 ） 提供・要求されるもの
temporal	object	いいえ	単純な有効期限以外の時間的制約
transaction	object	はい	ローカル遵守のための法的枠組み
reputation	object	いいえ	信頼性をローカルにどう解釈するか
disclosure	object	いいえ	段階的開示の設定
trust_generation	object	いいえ	低信頼環境での信頼ブートストラップ
culture	object	いいえ	交換規範の宣言
metadata	object	いいえ	拡張可能なキー・バリュータグ、注記
signature	string	はい	シグナルの内容（ signature を除く ）に対するデジタル署名

2.1 位置ブロック

不透明性を保ちつつ物流を可能にする。

```
"location": {
  "coarse": "ウップランズ・ヴェスビー", // 必須、人間可読
  "precise": {"lat": 59.5, "long": 17.9}, // 任意、非開示時は null
  "pickup": true,
  "delivery": false
}
```

2.2 リソースブロック

```
"resource": {
  "type": "food", // 主要カテゴリ（ §9 参照 ）
  "subtype": "トマト", // 自由記述またはコミュニティサブタイプ
  "quantity": 30,
  "unit": "kg",
  "quality": "有機", // 任意： "conventional", "surplus", "imperfect"
```

```
"tags": ["ビーガン", "グルテンフリー"] // 任意のフォークソノミー
}
```

2.3 時間ブロック

```
"temporal": {
  "availability_start": "2026-05-09T08:00:00Z",
  "availability_end": "2026-05-11T18:00:00Z",
  "needed_by": null, // リクエストの場合
  "recurrence": { // 任意
    "frequency": "weekly",
    "end_date": "2026-12-31",
    "occurrences": 10
  }
}
```

2.4 取引ブロック

社会的・法的枠組みを符号化。すべてのタイプは自己宣言。

```
"transaction": {
  "type": "gift", // "gift", "barter", "mutual_credit", "non_profit", "hybrid"
  "legal_entity": "org_num_123456-1234", // 任意
  "currency_accepted": null, // ハイブリッドの場合、例 "SEK"
  "vat_applicable": false,
  "receipt_required": false,
  "jurisdiction": "SE" // 自己宣言
}
```

2.5 評判ブロック（任意）

ノードが自身のシグナルがローカルでどう扱われるかに影響を与えたい場合のみ含める。グローバルな評判は維持されない。

```
"reputation": {
  "decay_rate": "14d", // デフォルト：14日ごとに半減
  "dimensions": ["food_handling", "reliability"]
}
```

2.6 開示ブロック（任意）

精度の段階的解放を制御。

```
"disclosure": {
  "current_level": 0, // 0 = 粗位置のみ; 1 = 正確な位置; 2 = 検証済み ID
  "thresholds": {
    "level_1": 5, // 成功したフルフィルメント 5 回後
    "level_2": 10
  },
  "consent_required": true,
}
```

```
"consent_timeout": "24h"
}
```

2.7 信頼生成ブロック（任意）

低信頼環境向け。

```
"trust_generation": {
  "vouches": [
    {
      "from_node": "Base58voucher",
      "domain": "food",
      "weight": 0.8,
      "expires": "2026-08-01T00:00:00Z"
    }
  ],
  "collateral": {
    "type": "labor_pledge",
    "amount": 10,
    "unit": "hours",
    "description": "コミュニティガーデンの維持",
    "validated_by": null
  }
}
```

2.8 文化ブロック（任意）

摩擦を防ぐために局所的な交換規範を宣言。

```
"culture": {
  "exchange_norm": "gift_with_reciprocity_expected",
  "notes": "私たちのコミュニティでは、同等の返礼が慣例ですが強制ではありません。"
}
```

2.9 メタデータブロック（任意）

```
"metadata": {
  "notes": "今朝収穫。容器をご持参ください。",
  "urgency": "medium"
}
```

3. 主要シグナルタイプ

3.1 オファー – 余剰または能力の宣言

オファーは完全なエンベロープを使用する。 `resource` および `location` ブロックが必須。オファーには時間制約（ `temporal` ）や繰り返し（ `recurrence` ）を含められる。

例

```
{
  "schema_version": "0.2",
  "signal_type": "offer",
  "id": "d290f1ee-6c54-4b01-90e6-d701748f0851",
  "node_id": "12D3KooWQkG9RN4nQJ6...",
  "timestamp": "2026-05-09T08:00:00Z",
  "expires": "2026-05-11T14:00:00Z",
  "location": {
    "coarse": "ウップランズ・ヴェスビー",
    "precise": null,
    "pickup": true,
    "delivery": false
  },
  "resource": {
    "type": "food",
    "subtype": "トマト",
    "quantity": 30,
    "unit": "kg",
    "quality": "有機",
    "tags": ["固定種", "ソース用トマト"]
  },
  "transaction": {
    "type": "gift",
    "legal_entity": "REKO_Väsby_1"
  },
  "reputation": {
    "decay_rate": "14d",
    "dimensions": ["food_handling"]
  },
  "metadata": {
    "notes": "今朝収穫。容器をご持参ください。"
  },
  "signature": "0x4a8b2c..."
}
```

3.2 リクエストー必要の宣言

リクエストはオファ어의構造を反映する。部分的な充足許容度のための `buffer` フィールドを含められる。

```
"buffer": {
  "min_quantity": 25,
  "max_quantity": 30
}
```

3.3 フルフィルメントー結果記録

フィードバックループ。フルフィルメントはオファ어とリクエストを参照し、結果を報告する。任意だが強く推奨——恒久的スコアを作らずにローカル学習を助ける。

```
{
  "schema_version": "0.2",
  "signal_type": "fulfillment",
  "id": "uuid-fulfillment-1",
  "node_id": "Base58nodeA",
  "timestamp": "2026-05-09T14:30:00Z",
  "references": {
    "offer_id": "uuid-offer-123",
    "request_id": "uuid-request-456"
  },
  "outcome": {
    "status": "completed",
    "quantity_fulfilled": 30,
    "unit": "kg",
    "reason_failure": null
  },
  "feedback": {                                // 任意
    "rating": 5,
    "note": "完全な状態！",
    "tags": ["時間通り", "フレンドリー"]
  },
  "signature": "..."}
}
```

3.4 ショートジアラート – システム生成の助言

ノードの分析機能によって生成され、持続的なギャップを可視化する。純粋に助言であり、人間の対話を促すためのもの。

```
{
  "schema_version": "0.2",
  "signal_type": "shortage_alert",
  "id": "uuid-alert-789",
  "node_id": "Base58analyticsNode",
  "timestamp": "2026-05-09T12:00:00Z",
  "resource": { "type": "food", "subtype": "冬野菜" },
  "period": { "start": "2026-11-01", "end": "2026-03-31" },
  "deficit_rate": 0.4,
  "current_production_intents": 2,
  "suggested_target": 5,
  "convening_suggestion": "バイオリージョンでの食の集いや REKO ミーティングを検討してください。",
  "scenario": {
    "if": { "new_intents": 3, "capacity_sq_m": 150 },
    "then": { "estimated_deficit_rate": 0.05 }
  },
  "signature": "..."}
}
```

4. 生産調整拡張

これらのシグナルにより、プロトコルは余剰分配を超えて**適応的生産計画**へと進むことができる——中央計画ではなく、会話に基づくコミットメントループである。

4.1 生産意図

ノードが何を生産する予定かを宣言する。

```
{
  "signal_type": "production_intent",
  "resource": { "type": "food", "subtype": "ケール" },
  "quantity": 100,
  "unit": "kg",
  "timeframe": "2026-Q3",
  "capacity_details": { "land_sq_m": 200, "method": "有機不耕起" }
}
```

4.2 キャパシティオファー

ノードが生産資産（土地、道具、労働）を提供する。

```
{
  "signal_type": "capacity_offer",
  "resource": { "type": "space", "subtype": "菜園区画" },
  "quantity": 50,
  "unit": "sq_m",
  "availability": "2026年3月～2026年10月",
  "constraints": "協同栽培者を希望"
}
```

4.3 協力リクエスト

生産目標のためのパートナーを探す。

```
{
  "signal_type": "collaboration_request",
  "goal": "冬野菜ギャップを埋める",
  "needed": { "type": "skill", "subtype": "農業" },
  "description": "ヴェスビー地区であと2名の栽培者を探しています。"
}
```

これら3つのシグナルが `shortage_alert` を実行可能な地域アジェンダに変える。プロトコルは「誰が意欲的か」「何が足りないか」を可視化し、人間がどうするかを決める。

5. 感謝 – 腐食しない感謝

私たちは意図的にポイント制の「ハート」システムを避ける。減衰と局所性があっても、可読的な感謝は最適化圧力、社会的ゲーム、指標追いを生む。

代わりに、感謝は二つの方法で表現される：

5.1 コモンズへの貢献

「ありがとう」という主な方法は、**ネットワークに何かを還元すること**である——個人に対してではなく、共有インフラに対して。`offer` シグナルは、それを触発した元のシグナルを参照する任意の `inspired_by` フィールドを含められる。

```
"inspired_by": "uuid-of-original-offer"
```

例：「学校給食室があまりに美味しいトマトを受け取ったので、コミュニティセンターの大掃除に4時間提供します。」

これにより、感謝は関係的で生成的、かつ数値化されないまま保たれる。

5.2 短命感謝シグナル（任意、制約付き）

直接の「ありがとう」が必要な場合、ノードは以下の必須制約付きで `appreciation` シグナルを発してよい：

- **短命**：24時間を超えて保存してはならない。
- **非希少**：数に制限なし。
- **ローカル可視**：発信元の信頼ゾーンを超えて連合されない。
- **アルゴリズム的に無視**：ボット、マッチングエンジン、評判モジュールは完全に無視しなければならない。

```
{
  "signal_type": "appreciation",
  "reference_signal": "uuid-of-fulfillment",
  "expression": "感謝",
  "note": "あなたの余剰が私たちの学校給食を素晴らしいものにしてくれました！",
  "ephemeral": true
}
```

これはデジタルな微笑みである。重みはなく、蓄積できない。

6. 文化的仲介

異なる交換規範背景を持つノードが交流するとき、摩擦は避けられない。プロトコルは任意の仲介スキャフォールドを提供する。

6.1 文化的デフォルト宣言（ノードメタデータ）

ノードはシグナルごとではなく、ノードプロファイルでデフォルト規範を宣言できる：

```
"cultural_default": {
  "exchange_norm": "gift_with_reciprocity_expected",
  "description": "私たちの伝統では、同等の返礼が慣例ですが強制ではありません。"
}
```

6.2 仲介シグナル

衝突が生じた場合、関係ノードのいずれかが発行できる：

```
{
  "signal_type": "cultural_mediation",
  "parties": ["nodeA", "nodeB"],
  "issue": "reciprocity_expectation_mismatch",
  "request": "seek_clarification",
  "mediator_suggestion": "some_bicultural_community_member"
}
```

6.3 仲介結果

仲介の結果は非拘束シグナルとして記録される：

```
{
  "signal_type": "mediation_outcome",
  "mediation_id": "...",
  "resolution": "agreed_to_clarify_expectations",
  "description": "NodeB は、贈与が純粋な贈り物である場合に明示的に表明します。"
}
```

仲介が失敗した場合、最終的な解決策は常に**退出**である——他ノードをミュートする、信頼ゾーンをフォークする、または離脱する。

7. プロトコルのガバナンス

憲法的原則は必要だが、明示的で最小限のガバナンス機構によって補完されねばならない。SSP を所有する単一の主体は存在しない。その進化は以下の規範に導かれる。

7.1 スキーマオーソリティ

- 変更は公に提案される（例：CC0 下の GitHub リポジトリ）。
- 実装は新しい `schema_version` を発することで採用を示す。
- 後方互換性のある拡張は奨励され、破壊的変更には新メジャーバージョンが必要。
- 決定はラフコンセンサスに従う。正式な投票機関は存在しない。

7.2 紛争解決

紛争は段階的に扱われる：

1. **ローカル信頼ゾーン** が内部で解決。
2. ゾーンを跨ぐ場合、文化的仲介を試行。
3. 未解決の場合、任意のノードがミュート、連合解除、またはフォークしてよい。

7.3 リレー運用

- クロスゾーンリレーはダムフォワードである。
- リレーは自身の `relay_policy` シグナルを公開しなければならない：

```
{
  "signal_type": "relay_policy",
```

```
"relay_type": "dumb_forwarder",
"filter_policy": "pass_through",
"logging_policy": "none"
}
```

- どのノードもリレーを運用できる。いかなるリレーも特別な地位を主張してはならない。
- ローカルノードはリレーの集中を監視する。ゾーン内のシグナルの 30% 超が単一リレーを通過する場合、注意喚起が発せられる。

7.4 フォーク閾値

フォークは第一級の権利である。ノードはメタデータの `protocol_variant` を通じてパリアントをシグナルできる。エンベロープと主要シグナルタイプが互換であり続ける限り、部分的な相互運用性は維持される。

8. 法的保護と退出機構

8.1 ノードレベルの法的メタデータ

各ノードはプロファイルに `legal` ブロックを含められる：

```
"legal": {
  "warrant_canary": "最終更新2026-05-01",
  "data_retention_days": 7,
  "right_to_delete": true,
  "jurisdiction_acceptance": "このノードはスウェーデン/EU法の下で運用されます"
}
```

ワラントカナリアは単なる日付であり、その不在（または更新停止）は強制開示の可能性を示唆するが、技術的な保証はない。

8.2 退出プロセス

- ノードは理由 `"exit"` を伴う `node_offline` シグナルをブロードキャストして退出する。
- その履歴シグナルはピアによって削除マークされる（最大 48 時間の猶予。現地法がより長い期間を要求する場合を除く）。
- ノードはピアにデータエクスポートを要求できる（実装依存）。
- グローバルなブラックリスト化は行われない。

9. リソースタイプ語彙（コア）

ノード間の分析を可能にしつつローカルな拡張を許すため、プロトコルは `resource.type` のための最小限の管理された語彙を定義する。サブタイプは自由形式のままでよい。

タイプ	説明	サブタイプ例
food	食用品	トマト、ケール、パン、蜂蜜
tools	物理的道具・装置	ドリル、ミシン
space	物理的空間	倉庫、作業場、菜園区画
transport	物資や人の移動	自転車、バン、配送枠
skill	人間の知識・労働	修理、教育、保育
energy	電力・熱	太陽光、薪、蓄電池
care	医療・高齢者・子育て支援	配食、話し相手
digital	デジタル財・サービス	ソフトウェア、3D プリント、デザイン

タイプレジストリは共同ウィキとして維持される。追加を門番する中央権威はなく、ローカルノードは拡張してよいが、未知のタイプは汎用親タイプにフォールバックする。

10. 連合トポロジー

- **信頼ゾーン内部**：ゴシッププロトコル、Matrix ルーム、Nostr リレーを介した完全 P2P メッシュ。各ノードは最近のシグナルのローカル短命ストアを保持する。
- **クロスゾーンルーティング**：リレー（ダムフォワード）を介したオプトイン。発見はグローバルではない。
- **グローバルレイヤー**：最終的に高度に集約されたコモンズレジストリ（リソースタイプウィキ、信頼ゾーンの連絡先）。これには**ライブシグナルは含まれない**。

11. 実装ガイダンスとパイロット設計

11.1 人間の処理を第一に

すべてのシグナルはプレーンテキスト（JSON またはコンパクト YAML）で表現可能でなければならない。人間が WhatsApp、Telegram、フォーラムに貼り付け、パーサーが取り込める。参照実装（Matrix ボット + ローカル SQLite ノード）が提供されるが、必須ではない。

11.2 AI 支援マッチング（任意、助言的）

ローカルノードは、マッチを提案し、供給過剰を予測し、生産カバレッジをシミュレーションする軽量モデル（量子化 LLM または制約ソルバー）を実行できる。すべての提案は上書き可能でなければならない。いかなるモデルも自律的に行動してはならない。

11.3 パイロット：デュアルパス展開

プロトコルの一般性を検証するため、二つの同時パイロットが推奨される：

- **パス A（信頼豊、例：スウェーデンの REKO リング）**：
 - WhatsApp/Matrix 上での基本機能（オファー/リクエスト/フルフィルメント）から開始。

- 人間の検証後にボット支援マッチングを導入。
 - 感謝、生産意図、文化的デフォルトを段階的にテスト。
 - 指標：調整効率、ユーザー受容、法的コンプライアンス。
- **パス B (低信頼、例：移民コミュニティハブ)：**
 - 困難な拡張から開始：保証、担保、段階的開示、文化的仲介。
 - 伝送手段として WhatsApp を使用；専用ハードウェア不要。
 - 目標：先行する信頼が存在しない場所でプロトコルが調整をブートストラップできることを証明する。
 - 指標：成功したフルフィルメント数、質的な信頼の出現、紛争解決パターン。

両パスの比較により、SSP が REKO アップグレードなのか、真の主権アーキテクチャなのかが判断される。

12. ライセンスとコントリビューション

SSP v0.2 は [Creative Commons Zero \(CC0\)](#) に基づきパブリックドメインに供される。実装はいかなるライセンスも使用してよい。プロトコルへの貢献は、カノニカルリポジトリ上のラフコンセンサスを通じて管理される。

この文書は生きた文法であり、最終的な教科書ではない。コミュニティがそれをテストし、壊し、適応させるにつれて成長する。森の菌糸ネットワークが機能するのは、シグナルが局所的、化学的、文脈依存的だからである。SSP は、人間の層において同様の疎さを目指す。